A REPORT

ON

DEMAND FORECASTING IN INVENTORY MANAGEMENT

BY

Dimpal Shetty **ID No.: 2024MT13052**

AT

Bengaluru, Karnataka

Airbus India Pvt Ltd, Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

November 2026

A REPORT

ON

DEMAND FORECASTING IN INVENTORY MANAGEMENT

BY

Dimpal Shetty **ID No.: 2024MT13052** **M.Tech. Software Systems**

Prepared in partial fulfilment of the
WILP Dissertation Course (S1-26_SSZG628T)

AT

Airbus India Pvt Ltd, Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

November 2026

Acknowledgements

I am grateful to my supervisor, Saurabh Mahajan, HO of Architecture, Airbus India Pvt Ltd, for the direction, time and honest feedback given at every stage of the project.

I thank my additional examiner, Karthik Mahadevan, Digital Sourcing Lead, Airbus India Pvt Ltd, for reviewing my work and for the helpful suggestions.

I thank my faculty mentor, Ashish Narang, BITS Pilani, for the guidance and support given throughout this dissertation.

I also thank my manager, PhaniKiran Vemagal, Airbus India Pvt Ltd, for supporting me in carrying out this work alongside my role.

Dimpal Shetty

2024MT13052

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
(RAJASTHAN)
WILP Division

Organization:	Airbus India Pvt Ltd	**Location:** Bengaluru
Duration:	13 weeks	**Date of Start:** 08 August 2026
Date of Submission:	10 November 2026	
Title of the Project:	Demand Forecasting in Inventory Management	
ID No./Name of the student:	2024MT13052 / Dimpal Shetty	
Name(s) and Designation(s) of Supervisor and Additional Examiner:	Saurabh Mahajan, HO of Architecture, Airbus India Pvt Ltd Karthik Mahadevan, Digital Sourcing Lead, Airbus India Pvt Ltd	
Name of the Faculty mentor:	Ashish Narang	
Key Words:	Spare-parts demand forecasting; LightGBM; ABC/XYZ classification; safety stock; Aircraft On Ground (AOG); landing gear; Apriori	
Project Areas:	Predictive Analytics; Machine Learning; Inventory Management	

Abstract: Airlines need the right spare parts ready before an aircraft comes in for repair. A missing part can ground the aircraft (Aircraft On Ground, AOG), while too much stock ties up money. Most parts are reordered on fixed schedules. This project builds a data-driven pipeline that forecasts demand for one landing-gear spare part and turns the forecast into a stock decision. It uses a public Kaggle dataset of 10,000 simulated landing records. A value and volatility (ABC/XYZ) classification picked the Landing Gear Actuator as the part worth forecasting. In this data, landing load, not runway condition, drives its failure. A tuned LightGBM model, tested only on batches it had never seen, cut the forecast error from 6.18 to 0.30 units per batch of 200 landings; Random Forest reached 0.10. A cost-based reorder rule then cut the safety buffer from about 23 units to 2, lowering the estimated yearly cost from Rs. 2.46 crore to Rs. 0.21 crore with no stockouts in the test. Apriori mining found parts that fail together, and a supplier dashboard shows the results. Because the data is simulated, the main contribution is the method; real airline data is needed to confirm the numbers.

Signature of Student
Date: 09-11-2026

Signature of Supervisor
Date: 09-11-2026

Contents

Acknowledgements **ii**

Abstract Sheet **iii**

List of Figures **vii**

List of Tables **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem statement 1

1.3 Objectives 2

1.4 Scope 2

1.5 Methodology in brief 2

2 Literature Review and Gap Analysis **4**

2.1 Spare-parts classification 4

2.2 Forecasting with usage data 4

2.3 Methods used in this project 5

2.4 Current practice and the gap 5

3 Dataset and Data Checks **7**

3.1 Source 7

3.2 Columns 7

3.3 Mapping flags to spare parts 8

3.4 Batches as planning periods 8

3.5 Data check: the failure flags follow fixed rules 8

4 System Design and Model Choice **10**

4.1 Architecture 10
4.2 Choice of forecasting model 11
4.3 How the forecast is built 11
4.4 Tools 12

5 Data Analysis, Baseline and Prioritisation 13
5.1 What drives actuator failure in this data 13
5.2 Which conditions relate to which part 14
5.3 Baseline forecast 15
5.4 Value and volatility (ABC/XYZ) classification 15
5.5 Companion parts (Apriori) 18
5.6 Cost of a stockout versus holding stock 19

6 Forecasting Model: Training and Tuning 21
6.1 Fair testing 21
6.2 Tuning 21
6.3 Results 22
6.4 Why Random Forest did slightly better 22
6.5 What the model relies on 24
6.6 Honest reading of these results 25

7 Reorder Engine and Cost Validation 26
7.1 The stock rule 26
7.2 Replaying the 50 batches 27
7.3 Order size (EOQ) 28
7.4 Assumptions behind the cost results 29

8 Supplier Dashboard 30

9 Limitations 32

10 Conclusions and Recommendations 33
10.1 Conclusions 33

10.2 Recommendations 34

A Settings Chosen in Each Fold **35**

B Forecasts for Every Batch **36**

References **38**

Glossary **40**

List of Figures

4.1 System architecture 10

5.1 Actuator failure rate by runway condition and landing load 13
5.2 Correlation of landing conditions with part failures 14
5.3 Baseline forecast against actual demand 16
5.4 ABC/XYZ classification matrix 17
5.5 Companion-part rules found by Apriori 19
5.6 AOG event cost against yearly holding cost 20

6.1 Forecast error by method 23
6.2 LightGBM forecast against actual demand 23
6.3 LightGBM feature importance 24

7.1 Yearly cost by method and scenario 27
7.2 Stock kept against actual demand 28

8.1 Supplier view of the dashboard 31

List of Tables

3.1 Dataset columns 7
3.2 Failure flags mapped to spare parts 8
3.3 Rules behind each failure flag 9

4.1 Model families considered 11

5.1 Correlation of landing conditions with part flags 15
5.2 ABC/XYZ classification of the four parts 16
5.3 Companion-part rules 18
5.4 AOG cost scenarios 19

6.1 Settings tried during tuning 22
6.2 Forecast error per batch 22

7.1 Results of replaying the 50 batches 27
7.2 Order size for different order costs 29

A.1 Settings chosen in each fold 35

B.1 Forecasts for every batch 36

1. Introduction

1.1 Background

Airline maintenance teams, known as MRO (Maintenance, Repair and Overhaul) teams, need the right spare parts on the shelf before an aircraft comes in for repair. If a part is missing, the aircraft cannot fly until the part arrives. This is called an AOG (Aircraft On Ground) event. Public industry figures put the cost of an AOG event at roughly US$10,000 to US$150,000 per hour, depending on the aircraft and the route.

The opposite mistake is also costly. Keeping many spare parts "just in case" ties up money in stock that sits on a shelf. For an expensive part, such as a landing-gear actuator, every extra unit held for a year costs a lot in capital, storage and insurance.

Today, most spare parts are reordered on a fixed schedule based on flight hours or flight cycles. A fixed schedule treats every aircraft the same. It does not look at how each aircraft was actually used, for example how hard it landed or how heavily it was loaded.

1.2 Problem statement

This project addresses the following problem: how can landing-by-landing operating data be used to forecast demand for a high-value landing-gear spare part, and to turn that forecast into a stock decision that balances the cost of running out against the cost of holding stock?

1.3 Objectives

1. Sort the landing-gear spare parts into priority groups by how much money they account for and how steady their demand is, to decide which part is worth a machine-learning forecast.
2. Build, tune and fairly test a model that forecasts how many units of the chosen part will be needed, using landing conditions as inputs, and compare it against a simple baseline and a second model.
3. Find which spare parts tend to be needed together, so a supplier can bundle them.
4. Turn the forecast into a stock and reorder recommendation in Indian Rupees, weighing the cost of an AOG event against the cost of holding stock.
5. Show the result in a simple dashboard a supplier can use.

1.4 Scope

The project forecasts demand for one part in detail: the Landing Gear Actuator, chosen by the classification in Chapter 5. It uses one public dataset of 10,000 landing records (Chapter 3). Live flight data, weather feeds and real Airbus production data are outside the scope. Two companion parts named in the original project outline, seal kits and structural bolts, are not in the dataset and so are not covered.

1.5 Methodology in brief

The work was carried out in seven phases:

1. Gap analysis and system design.
2. Loading and exploring the data.

3. Baseline forecast, value/volatility classification and companion-part mining.
4. Midterm report.
5. Training and tuning the forecasting models (LightGBM and Random Forest).
6. Building the reorder engine and checking it against the cost model.
7. Building the supplier dashboard and writing this report.

All analysis was done in Python. Every number in this report was produced by the project's Python scripts and can be reproduced by running them.

2. Literature Review and Gap Analysis

2.1 Spare-parts classification

Spare-parts classification and demand forecasting is a well-studied area. Bacchetti and Saccani [1] reviewed the field and found a gap between research and practice: many published methods are tested on idealised data and are rarely checked against real maintenance records. Lagoda and Klumpp [2] proposed a modified ABC-XYZ analysis that links the classification step to the forecasting step instead of treating them separately. This project follows the same idea: the classification result is used to choose which part to forecast. Nafiurridha and Choiruddin [3] used supervised machine learning to classify aircraft spare parts. This project keeps the classification simple and easy to explain (ABC/XYZ), and uses machine learning only for the forecast.

2.2 Forecasting with usage data

Olmo and Domingo [4] proposed a framework for forecasting demand for aerospace rotable parts. Their main argument is that fleet usage data captures differences in how aircraft are operated, which fixed schedules miss. LightGBM was their best model for most parts. Stanton et al. [5] compared hand-chosen machine-learning models with automated machine learning (AutoML) on landing-gear data. This supports comparing more than one model on a landing-gear part, rather than picking one without a test.

2.3 Methods used in this project

The Apriori algorithm of Agrawal and Srikant [6] is the standard method for finding items that occur together, and is used here for companion parts. LightGBM [7] is a fast gradient-boosting library that builds decision trees one after another, each correcting the errors of the previous ones. Grinsztajn et al. [8] showed that tree-based models usually beat deep learning on small and medium tabular datasets like the one used here. Random Forest [9], which averages many independent decision trees, is used as the comparison model. For stock decisions, this project uses the newsvendor rule for safety stock and the economic order quantity (EOQ) for order size, both standard methods in inventory management [10, 11].

2.4 Current practice and the gap

In current practice:

- demand is forecast from a fixed flight-hour or flight-cycle schedule, regardless of how each aircraft was used;
- parts are ranked by yearly cost only, not by how steady their demand is;
- safety stock is a fixed buffer that is rarely updated;
- parts are bundled in orders based on engineers' experience, not on measured patterns.

Olmo and Domingo [4] is the closest study to this one. Compared with that study, this project:

1. adds a prioritisation step, so that only the part worth forecasting is forecast;
2. measures which operating condition drives failure, with a check that the result does not depend on runway condition;
3. adds companion-part mining for bundled orders;
4. goes all the way to a cost-based stock and order decision and a supplier dashboard.

A clear weakness compared with that study is the data: this project uses a small public simulated dataset, while that study used real fleet data.

3. Dataset and Data Checks

3.1 Source

The dataset is a public landing-gear maintenance dataset from Kaggle [12]. The downloaded file is named `landing_gear_maintenance_simulated_data.xlsx`. It has 10,000 rows and 12 columns, with no missing values. Each row is one landing. There is no date and no aircraft ID, so landings cannot be linked into the history of one aircraft.

3.2 Columns

Table 3.1 lists the columns. Eight describe the conditions of the landing. Four are yes/no flags that show whether a component showed a failure or wear sign on that landing.

Table 3.1: Dataset columns

Column	Meaning
`landing_cycles`	Number of landing cycles the aircraft had built up
`load_during_landing`	Load on the landing gear at touchdown
`taxiing_duration`	How long the aircraft taxied
`speed_during_landing`	Speed at touchdown
`braking_force`	How hard the brakes were applied
`tire_pressure`	Tyre pressure at landing
`hydraulic_pressure`	Hydraulic system pressure
`runway_condition`	Dry, wet or snowy
`brake_degradation`	1 if the brake showed wear, else 0
`shock_absorber_failure`	1 if the shock absorber failed, else 0
`tire_wear`	1 if the tyre showed wear, else 0
`hydraulic_system_failure`	1 if the hydraulic system failed, else 0

3.3 Mapping flags to spare parts

Each flag was mapped to one spare part (Table 3.2). This mapping is the author's own choice; it is not given in the data. The rates do not add up to 100% because each landing has four separate yes/no checks, so one landing can flag several parts. For example, 5,689 landings flag both the actuator and the wheel unit.

Table 3.2: Failure flags mapped to spare parts

Flag in data	Spare part (author's mapping)	Flag rate
`brake_degradation`	Brake Assembly	95.3%
`shock_absorber_failure`	Landing Gear Actuator	59.7%
`tire_wear`	Wheel Unit	81.7%
`hydraulic_system_failure`	Hydraulic Line	24.5%

3.4 Batches as planning periods

Because the data has no dates, the 10,000 landings were split, in file order, into 50 batches of 200 landings each. Each batch stands in for one planning period. Under the assumed fleet use of 2,000 landings per year, one batch is about one-tenth of a year (about 36 days). The number of actuator flags in a batch is the "demand" for that period. The same batches are used everywhere in this report.

3.5 Data check: the failure flags follow fixed rules

During model training (Chapter 6) the models became almost perfect. This is unusual for real maintenance data, so the data was checked. A small decision tree with only two splits was fitted to each flag. For every flag, a simple two-part rule matched all 10,000 rows exactly (Table 3.3). Each

rule was then checked directly against the data.

Table 3.3: Exact rules behind each failure flag (each matches 10,000 of 10,000 rows)

Part	**Flag = 1 when**
Landing Gear Actuator	load during landing > 70 **or** landing cycles > 401
Wheel Unit	braking force > 40 **or** landing cycles > 300
Brake Assembly	braking force > 35 **or** landing cycles > 351.5
Hydraulic Line	hydraulic pressure < 2800 **or** landing cycles > 450.5

This shows that the dataset is simulated: the flags were produced by rules, not observed on real aircraft. This has three effects, which are kept in mind throughout the report:

1. Runway condition is not part of any rule. So the finding that "landing load matters and weather does not" (Chapter 5) describes how the simulator was built. It is not proof about real aircraft.
2. The forecasting models can learn these rules exactly, so their errors are close to zero. Real data would contain randomness and the errors would be larger.
3. The value of this project is the *method*: the full pipeline from data to stock decision. Real airline data is needed to confirm the size of the benefit.

The dataset was kept, after discussion, because the pipeline itself does not depend on the data source and can be re-run on real data without changes.

4. System Design and Model Choice

4.1 Architecture

Figure 4.1 shows how data flows through the system. The main path is always used: the data is prepared, the classification picks the part to forecast, the forecast model predicts demand, and the cost and reorder engine turns the forecast into a stock recommendation shown on the supplier dashboard. Companion-part mining is an optional side branch, shown when a supplier wants bundling suggestions.

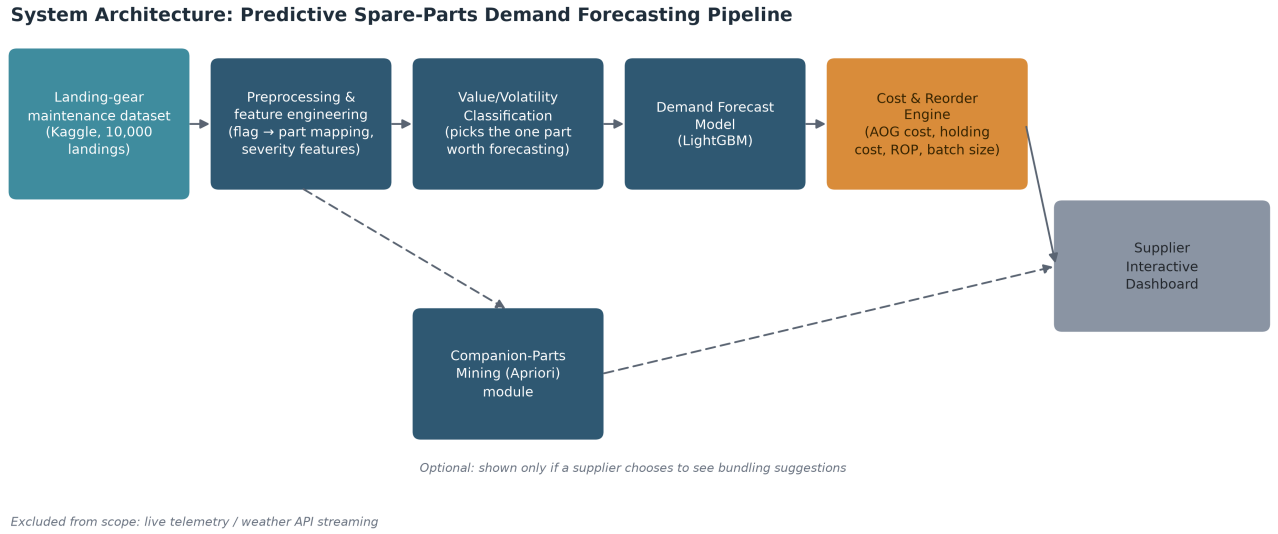


Figure 4.1: System architecture. The solid path is always used; the dashed branch (companion parts) is optional.

4.2 Choice of forecasting model

Table 4.1 compares the model families considered.

Table 4.1: Model families considered

Model family	How it works	Decision
Time-series models (ARIMA, Prophet)	Learn trend and season from a demand series over time.	Not used. The data has no dates, so there is no time series to learn from.
Deep learning (LSTM, Transformer)	Neural networks that learn from ordered sequences.	Not used. There is no aircraft ID or date to form sequences, and tree models usually win on small tabular data [8].
Random Forest	Averages many decision trees built independently.	Used as the comparison model.
LightGBM (gradient boosting)	Builds trees one after another, each fixing the errors of the last.	Chosen as the main model: handles the runway category directly, trains fast, and gives feature importance [7]. It was the best model in the closest study [4].

LightGBM was chosen as the main model before any training was done, based on the reasons above. Random Forest was trained and tested in exactly the same way so that the choice could be checked.

4.3 How the forecast is built

There are only 50 batches, which is too few rows to train a model directly on batch totals. Instead, the model is trained on all 10,000 individual landings. For each landing, it predicts the *chance* that the actuator flag is 1, using only the eight landing conditions. The other three failure flags are *not* used as inputs, because on a real day they would not be known in advance. The chances for the 200 landings in a batch are then added up. This sum is the forecast number of actuators needed in that batch.

4.4 Tools

The project uses Python 3.12 with pandas for data handling, scikit-learn [13] for Random Forest and testing, LightGBM 4.6.0, mlxtend [14] for Apriori, and matplotlib for charts. The dashboard is a single HTML file with no outside libraries, so it works offline.

5. Data Analysis, Baseline and Prioritisation

5.1 What drives actuator failure in this data

Figure 5.1 compares the actuator flag rate by runway condition and by landing load. Runway condition barely changes the rate: 59.6% on dry runways, 60.9% on wet and 58.6% on snowy. Landing load changes it a lot: the rate rises from near 0% for the lightest landings to near 100% for the heaviest.

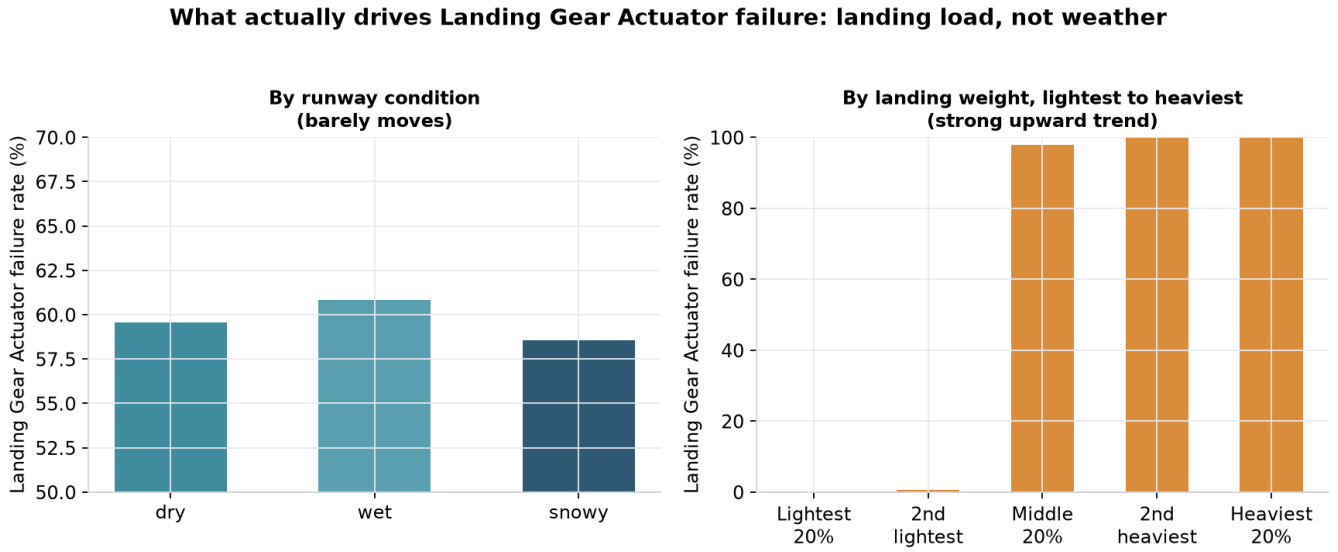


Figure 5.1: Actuator flag rate by runway condition (left) and by landing load, lightest to heaviest (right).

Robustness check. Load might only look important because heavy landings happen more on wet or snowy runways. To test this, the link between load and actuator failure (correlation) was measured separately for each runway condition. It was 0.814 on dry, 0.823 on wet and 0.827 on snowy runways. Since the link is the same in all three groups, runway condition is not hiding behind the load effect.

As explained in Section 3.5, this result reflects how the simulator was built: runway condition is not in any of its rules. The check is still correct, but it confirms the simulator, not real aircraft behaviour.

5.2 Which conditions relate to which part

Figure 5.2 and Table 5.1 show the correlation between each landing condition and each part's flag. A value near 1 or −1 means a strong link; a value near 0 means no link.

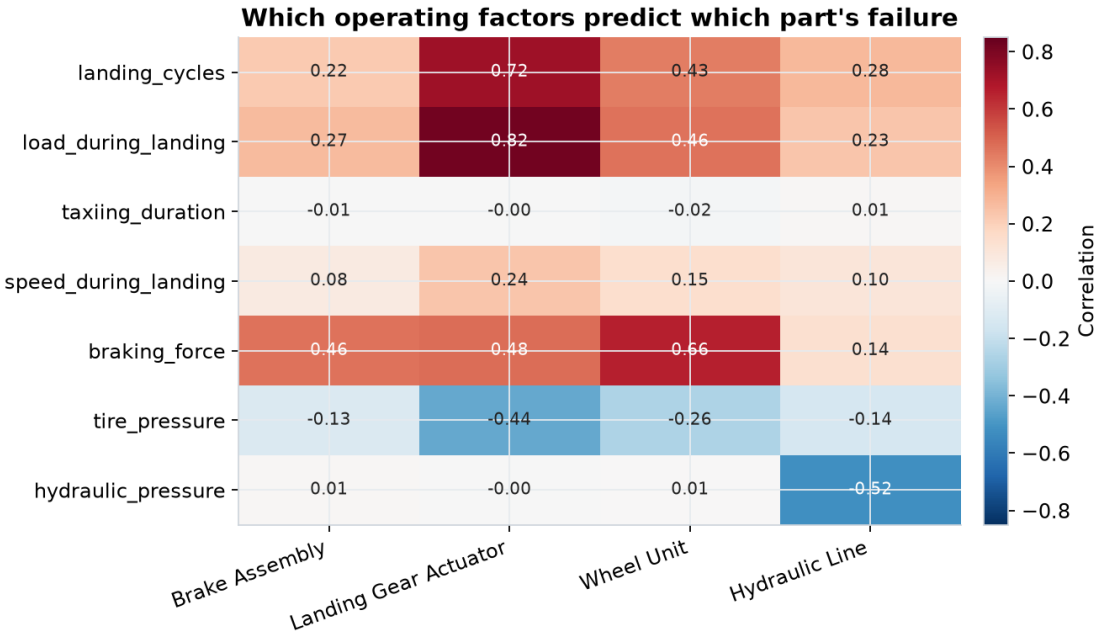


Figure 5.2: Correlation between each landing condition and each part's failure flag.

Load and braking force are the strongest links for the actuator, wheel unit and brake assembly. For the hydraulic line, the strongest link is hydraulic pressure. For the actuator, landing cycles (0.72) are only a little weaker than load (0.82), so both matter. All brake assembly links are weak, likely because it is flagged 95% of the time and so varies very little. Taxiing duration has no link to any part.

Table 5.1: Correlation of each landing condition with each part's flag

Condition	Brake Assy.	Actuator	Wheel Unit	Hydr. Line
Landing cycles	0.22	0.72	0.43	0.28
Load during landing	0.27	**0.82**	0.46	0.23
Taxiing duration	−0.01	0.00	−0.02	0.01
Speed during landing	0.08	0.24	0.15	0.10
Braking force	**0.46**	0.48	**0.66**	0.14
Tyre pressure	−0.13	−0.44	−0.26	−0.14
Hydraulic pressure	0.01	0.00	0.01	**−0.52**

5.3 Baseline forecast

A baseline is a simple forecast that any model must beat. Here, the baseline always predicts the average number of actuator flags per batch, which is 119.44. Its mean absolute error (MAE), the average size of the miss, is 5.52 units per batch (Figure 5.3). This average included the batch being predicted, which makes the error look slightly smaller than it really is. The fairer version used in Chapter 6, which averages only the training batches, has an error of 6.18.

5.4 Value and volatility (ABC/XYZ) classification

The ABC/XYZ classification ranks parts in two ways. ABC ranks parts by yearly value (number needed per year × unit cost): A is the highest value, C the lowest. XYZ ranks parts by how steady their demand is, measured by the coefficient of variation (CV), which is the spread of demand divided by its average. X is the steadiest, Z the most erratic. Standard cut-offs were used: 80%/95% of cumulative value for A/B/C, and CV of 0.15/0.30 for X/Y/Z.

Unit costs are assumptions based on public price ranges, converted at Rs. 96 per US dollar. Yearly quantities assume 2,000 landings per year. The values are therefore priority scores, not real spending forecasts.

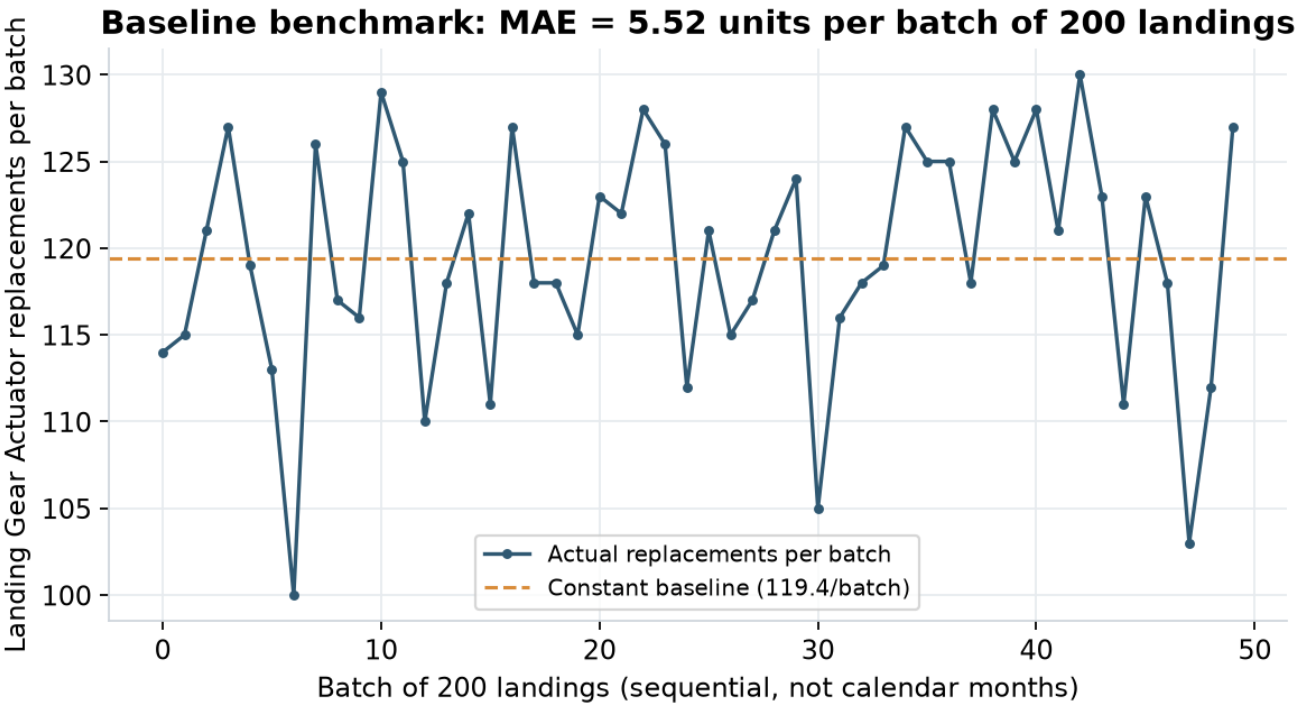


Figure 5.3: Actuator flags per batch of 200 landings compared with the constant-average baseline.

Table 5.2: ABC/XYZ classification of the four parts

Part	Flag rate	Unit cost (Rs.)	Value (Rs. cr/yr)	CV	Class
Landing Gear Actuator	59.7%	43,20,000	516.0	0.06	A/X
Brake Assembly	95.3%	17,28,000	329.3	0.01	B/X
Wheel Unit	81.7%	5,76,000	94.1	0.03	C/X
Hydraulic Line	24.5%	2,40,000	11.8	0.12	C/X

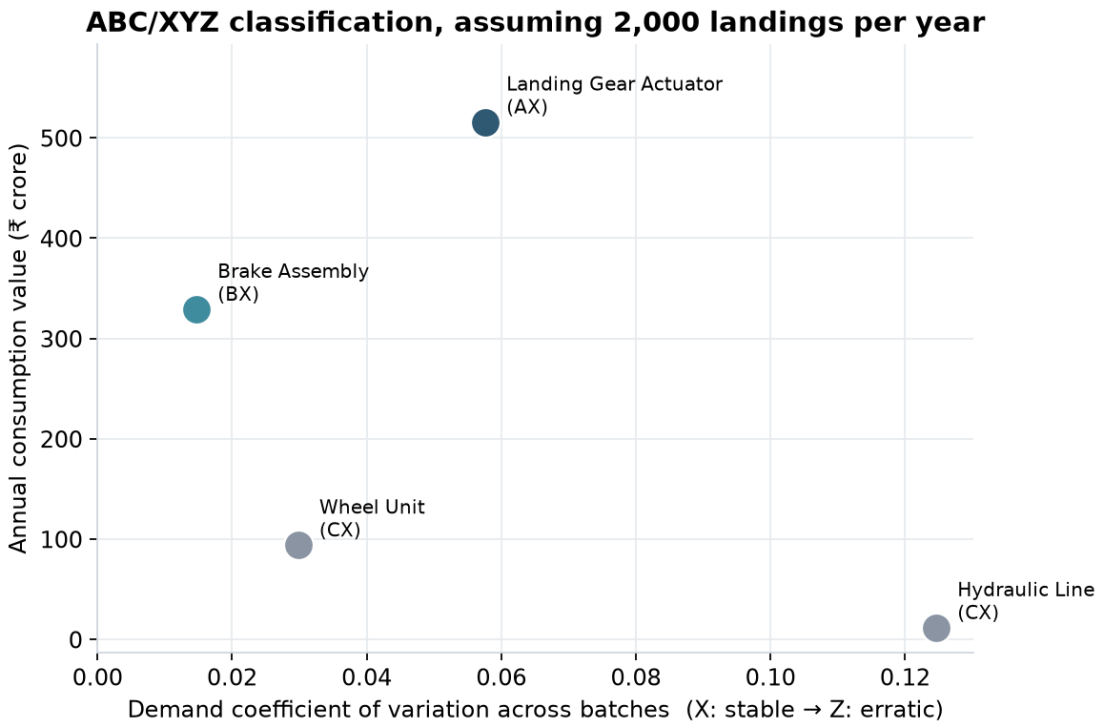


Figure 5.4: ABC/XYZ matrix. The Landing Gear Actuator is the only Class A/X part.

The Landing Gear Actuator is the only Class A/X part (Table 5.2, Figure 5.4): it has the highest value and steady demand. It was therefore chosen as the part to forecast. It was picked because of its value, not because it fails most often; the brake assembly is flagged more often (95.3%). The original project outline had assumed the brake assembly would be the main part. The classification changed that choice.

5.5 Companion parts (Apriori)

The Apriori algorithm [6] was used to find parts that are flagged together on the same landing. Three measures describe each rule "when part X is flagged, part Y is also flagged":

- **support**: the share of all landings where both parts are flagged;
- **confidence**: of the landings where X is flagged, the share where Y is also flagged;
- **lift**: how many times more often Y appears with X than it would by chance (above 1 means a real link).

With minimum support 0.15 and minimum lift 1.05, three useful rules were found (Table 5.3, Figure 5.5).

Table 5.3: Companion-part rules found by Apriori

Rule	Support	Confidence	Lift
Hydraulic Line → Landing Gear Actuator	0.18	0.72	1.20
Landing Gear Actuator → Wheel Unit	0.57	0.95	1.17
Hydraulic Line → Wheel Unit	0.21	0.87	1.07

The most useful rule for a supplier is the second one: when the actuator is flagged, the wheel unit is also flagged 95% of the time. These links partly come from how the simulator's rules were written (Table 3.3): landing cycles appears in every rule, and load and braking force are themselves linked (correlation 0.58). On real data, the rules may be weaker or stronger.

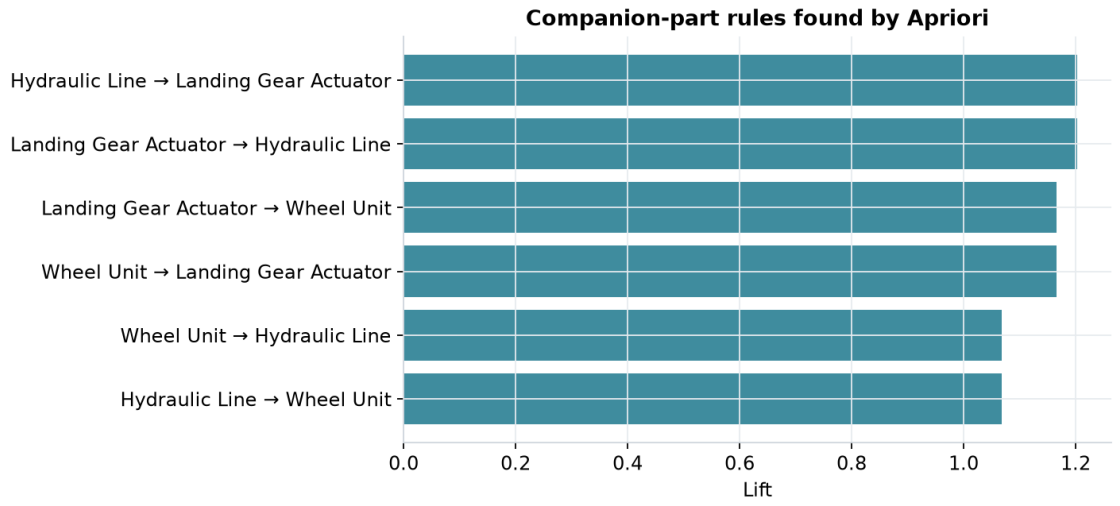


Figure 5.5: Companion-part rules found by Apriori, ranked by lift.

5.6 Cost of a stockout versus holding stock

Table 5.4 shows the cost of one AOG event under three assumed scenarios, each lasting 18 hours.

Table 5.4: Assumed AOG cost scenarios (18-hour event, Rs. 96 = US$1)

Scenario	**AOG cost per hour (Rs.)**	**Cost of one event (Rs.)**
Low (narrow-body, domestic)	9,60,000	1,72,80,000
Mid (narrow-body, typical)	43,20,000	7,77,60,000
High (wide-body, peak)	1,44,00,000	25,92,00,000

The holding cost is assumed to be 25% of the part's price per year. For one actuator (Rs. 43,20,000), this is Rs. 10,80,000 per year. Even the Low scenario costs about 16 times that amount for a single event. This is why the stock rule in Chapter 7 aims to avoid stockouts with very high probability.

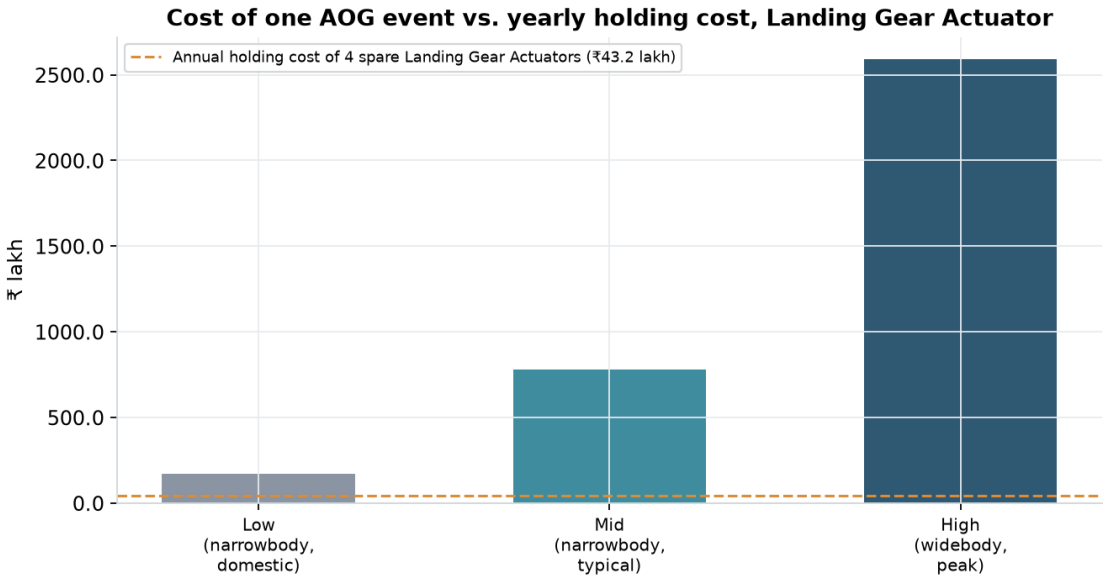


Figure 5.6: Cost of one AOG event in each scenario compared with the yearly holding cost used in the midterm example.

6. Forecasting Model: Training and Tuning

6.1 Fair testing

A model must be tested on data it has never seen, otherwise its error looks smaller than it really is. This project uses cross-validation over batches:

1. The 50 batches are split at random into 5 groups ("folds") of 10 batches.
2. The model is trained on the landings in 4 folds (40 batches) and tested on the 5th fold (10 batches) it has never seen.
3. This is repeated 5 times, so every batch is tested exactly once by a model that did not see it.

Whole batches are kept together, so no landing from a test batch is ever used in training.

6.2 Tuning

Hyperparameter tuning means trying different model settings and keeping the best one. To keep the test fair, tuning was done *only* inside the training batches of each fold. The training batches were split again into 4 inner folds; each setting was trained on 3 and scored on the 4th, using log loss (a score that punishes confident wrong predictions). The setting with the lowest average log loss was then trained on all training batches and used on the test fold. The settings tried are in Table 6.1. The grids were kept small and standard; they were not adjusted after seeing the results.

LightGBM chose 7 leaves, learning rate 0.03 and 300 trees in four of the five folds (15 leaves in the other). Random Forest chose minimum leaf size 1 and 60% of features in all five folds. The full list is in Appendix A.

Table 6.1: Settings tried during tuning

Model	Settings tried
LightGBM (12 combinations)	number of leaves {7, 15, 31}; learning rate {0.03, 0.1}; number of trees {100, 300}; minimum samples per leaf 20
Random Forest (6 combinations)	300 trees; minimum samples per leaf {1, 5, 20}; features tried per split {square root, 60%}

6.3 Results

Table 6.2 and Figure 6.1 compare the forecast error per batch.

Table 6.2: Forecast error per batch of 200 landings (lower is better)

Method	MAE (units per batch)
Midterm baseline (average of all 50 batches)	5.52
Fair baseline (average of training batches only)	6.18
Random Forest (tuned, tested on unseen batches)	**0.10**
LightGBM (tuned, tested on unseen batches)	**0.30**

Both models reduce the error from about 6 units to well under 1 unit per batch (Figure 6.2). At the level of single landings, both models separate failures from non-failures almost perfectly: the AUC (a score from 0.5 for guessing to 1.0 for perfect) is 1.000 for both, and the log loss is 0.0053 for LightGBM and 0.0008 for Random Forest.

6.4 Why Random Forest did slightly better

Random Forest was slightly more accurate than LightGBM (0.10 against 0.30). The likely reason is the sharp cut-off in the simulator's rule (load > 70 or cycles > 401). Random Forest's fully grown trees match a sharp cut-off exactly. LightGBM, with its small learning rate, gives slightly softer chances close to the cut-off. The gap is less than one unit per 200 landings, so it makes no practical difference. LightGBM is kept as the main model because it was chosen in advance for the reasons

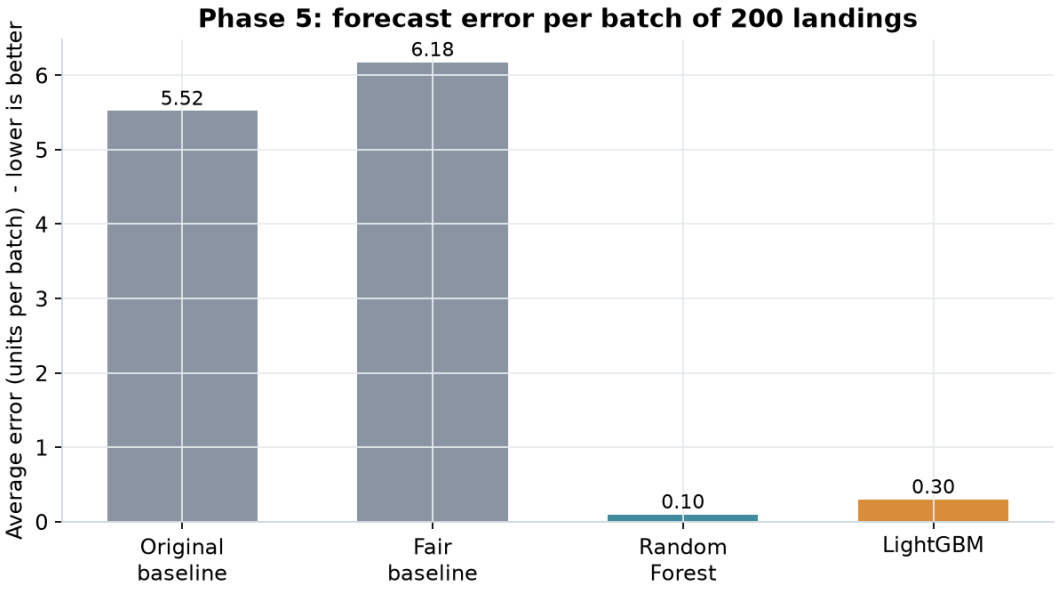


Figure 6.1: Forecast error (MAE) per batch for each method.

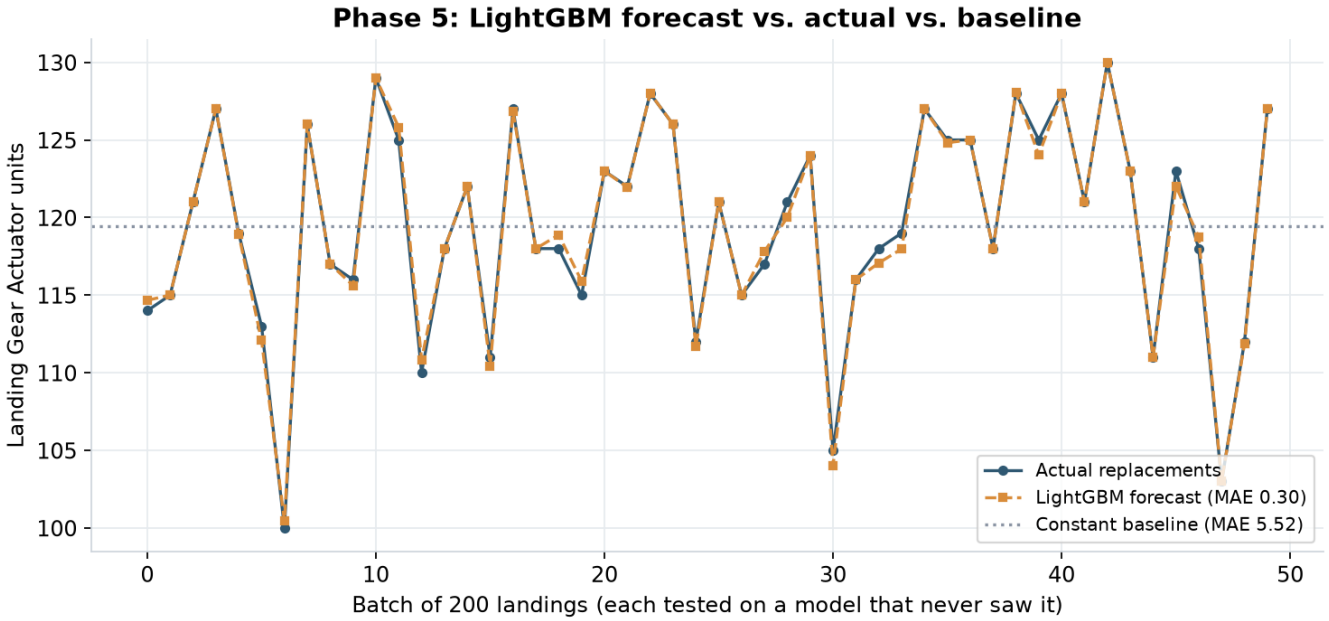


Figure 6.2: LightGBM forecast compared with actual actuator flags per batch, and the constant baseline. Each batch was forecast by a model that never saw it.

in Table 4.1, and because on real, noisier data gradient boosting usually performs well [4]. This choice should be checked again on real data.

6.5 What the model relies on

Feature importance shows how much each input helped the model. For LightGBM (Figure 6.3), load during landing accounts for 99.1% of the total gain and landing cycles for 0.7%. Every other condition is close to zero, and runway condition is 0.0%.

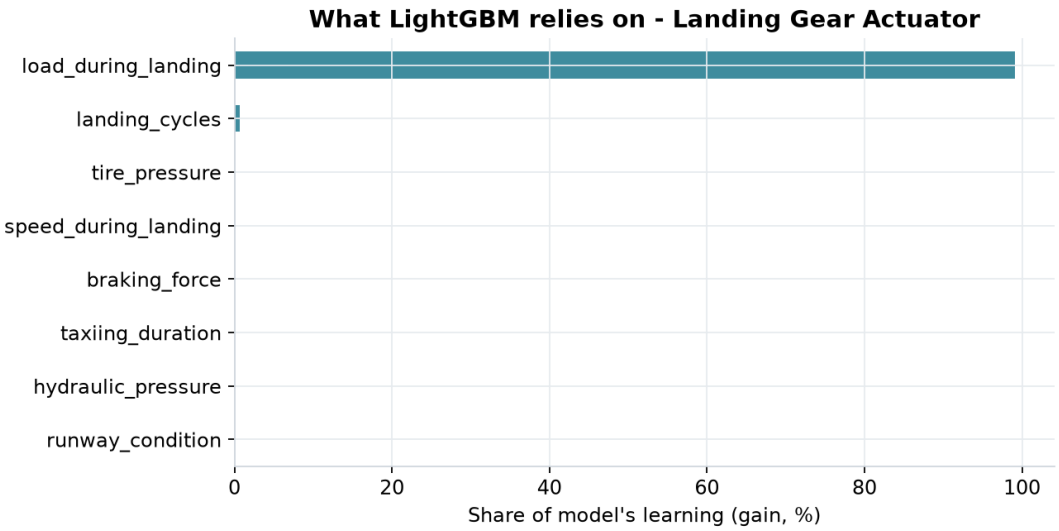


Figure 6.3: LightGBM feature importance (share of total gain).

This matches the rule in Table 3.3: the model found the two conditions the simulator uses and ignored the rest. Load dominates because the two limits overlap: of the 1,982 landings with more than 401 cycles, 1,966 also have load above 70. Only 16 of the 5,972 actuator failures are explained by cycles alone.

6.6 Honest reading of these results

Errors this small are only possible because the flags follow an exact rule with no randomness (Section 3.5). The best possible error on this data is zero. The results show that the pipeline works correctly and that the fair-testing setup does not leak information. They do *not* show how accurate the model would be on real aircraft.

7. Reorder Engine and Cost Validation

7.1 The stock rule

Before each batch starts, the shelf is filled up to a target level:

$$\text{target stock} = \text{forecast} + z \times \text{typical forecast error}.$$

The second part is the safety buffer. The "typical forecast error" is the spread (standard deviation) of each method's past errors, measured only on the *other* folds, never on the batch being planned.

The factor z comes from the newsvendor rule [10]. It compares two costs:

- cost of being one unit short = one AOG event (Table 5.4);
- cost of one extra unit on the shelf for one batch = Rs. 43,20,000 × 25% × 0.1 year = Rs. 1,08,000.

The target chance of *not* running out, the service level, is:

$$\text{service level} = \frac{\text{cost of being short}}{\text{cost of being short} + \text{cost of one extra unit}}.$$

This gives service levels of 99.38%, 99.86% and 99.96% for the Low, Mid and High scenarios, so z = 2.50, 2.99 and 3.34. Because an AOG event costs so much more than holding a part, the rule aims to have stock almost all the time.

7.2 Replaying the 50 batches

The 50 batches were replayed for three methods: the fair baseline, LightGBM and Random Forest. For each batch, the target stock was compared with the actual demand. Units short were counted as AOG events, and spare units were charged at the holding cost. Results are in Table 7.1 and Figure 7.1.

Table 7.1: Replay of the 50 batches: safety buffer, shortages and yearly cost

Scenario	Method	Avg. safety buffer	Batches short	Yearly cost (Rs.)
Low	Baseline	19.0 units	0 of 50	2.05 crore
	LightGBM	1.9 units	0 of 50	20.3 lakh
	Random Forest	1.0 unit	0 of 50	10.4 lakh
Mid	Baseline	22.8 units	0 of 50	2.46 crore
	LightGBM	2.0 units	0 of 50	21.4 lakh
	Random Forest	1.0 unit	0 of 50	10.4 lakh
High	Baseline	25.4 units	0 of 50	2.74 crore
	LightGBM	2.1 units	0 of 50	21.8 lakh
	Random Forest	1.0 unit	0 of 50	10.6 lakh

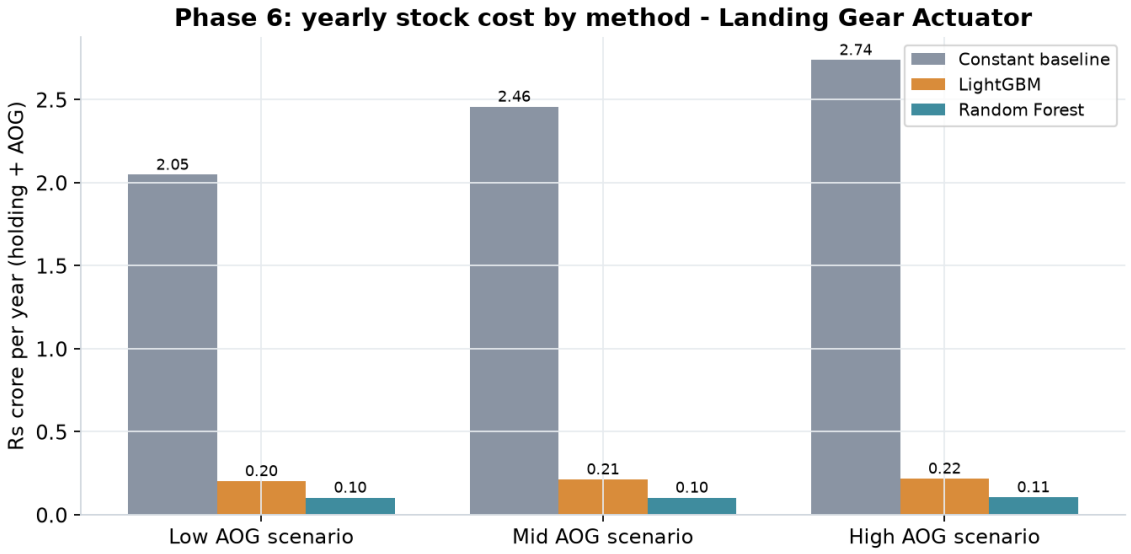


Figure 7.1: Yearly cost of each method under the three AOG scenarios.

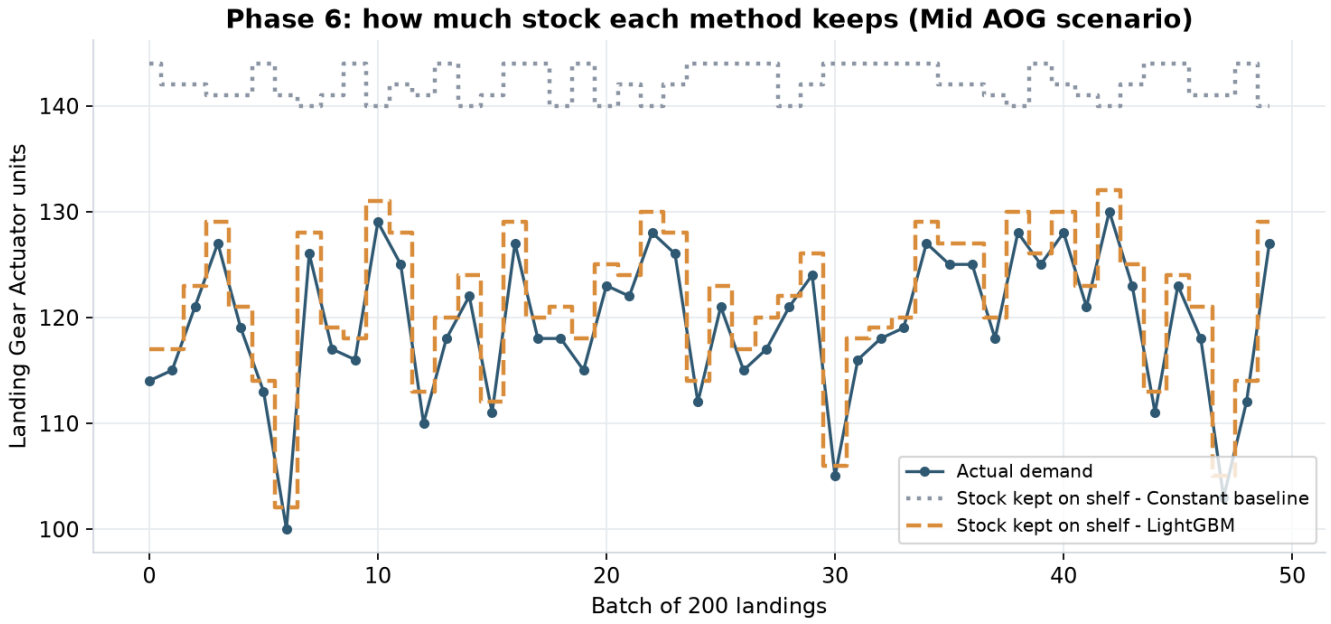


Figure 7.2: Stock kept on the shelf by each method compared with actual demand, Mid scenario.

No method ran short in any of the 50 batches. All of the saving therefore comes from holding fewer spare units for the same protection (Figure 7.2). With LightGBM, the safety buffer drops from about 23 units to about 2, and the yearly cost drops by about Rs. 1.84 crore (Low), Rs. 2.24 crore (Mid) and Rs. 2.52 crore (High).

Fifty batches is too few to test rare stockouts properly. With a target service level of 99.86%, a stockout would be expected less than once in 50 batches even if the rule works as designed. The "0 shortages" result is encouraging but not proof.

7.3 Order size (EOQ)

The economic order quantity (EOQ) is the number of units per order that balances the cost of placing orders against the cost of holding stock [11]:

$$\text{EOQ} = \sqrt{\frac{2 \times \text{yearly demand} \times \text{cost per order}}{\text{holding cost per unit per year}}}.$$

Yearly demand is about 1,194 actuators for every method, and the holding cost is Rs. 10,80,000 per unit per year. The cost of placing one order (paperwork, transport and inspection) is not known, so a range was tested (Table 7.2). Rs. 50,000 is the main assumption.

Table 7.2: Order size (EOQ) for different assumed order costs

Cost per order (Rs.)	EOQ (units)	Orders per year	Orders per batch
25,000	7	171	17
50,000	**11**	**109**	**11**
1,00,000	15	80	8
2,00,000	21	57	6

EOQ is the same whichever forecast is used, because all methods predict about the same yearly total. The forecast's value is in the safety buffer, not in the order size.

7.4 Assumptions behind the cost results

- 2,000 landings per year, so one batch is 0.1 year.
- Parts ordered for a batch arrive before that batch starts (lead time is covered by planning one batch ahead).
- Each unit short is one separate 18-hour AOG event.
- The forecast uses each landing's conditions. In real use, these would come from flight plans and known cycle counts.
- Demand is about 119 actuators per 200 landings, because the simulated flag rate is 60%. This is far higher than for real aircraft, and does not fit well with the "4 spare units" holding example used in the midterm report (Figure 5.6). This is a feature of the simulated data.

8. Supplier Dashboard

The results are shown in a dashboard built as a single HTML file. It needs no internet connection and no installation; it opens in any web browser. It is created by a script that reads the outputs of Chapters 6 and 7, and checks that its yearly costs match those in Table 7.1 before writing the file.

The dashboard has two parts.

1. Supplier view (Figure 8.1). This part shows only what a supplier needs, using the LightGBM forecast. There is no choice of model, because a supplier does not need one. The user picks a planning batch, an AOG cost scenario and an order cost. Six panels then show:

- **Forecast demand**: actuators expected in the batch;
- **Stock to keep**: forecast plus safety buffer;
- **Safety buffer**: extra units kept in case the forecast is low;
- **Order plan**: units per order (EOQ) and orders per batch;
- **What actually happened**: actual demand and whether stock was enough;
- **Ship together**: the part most often needed with the actuator (Wheel Unit, 95%).

Below these are a chart of stock kept against actual demand for all 50 batches, the list of companion-part rules, and a short "What the words mean" section.

2. For evaluation. This part is clearly marked as being for the report and viva, not for suppliers. It compares LightGBM, Random Forest and the baseline: forecast error, safety buffer, shortages and yearly cost.

A banner at the top and a list at the bottom state that the data is simulated and list every cost assumption.

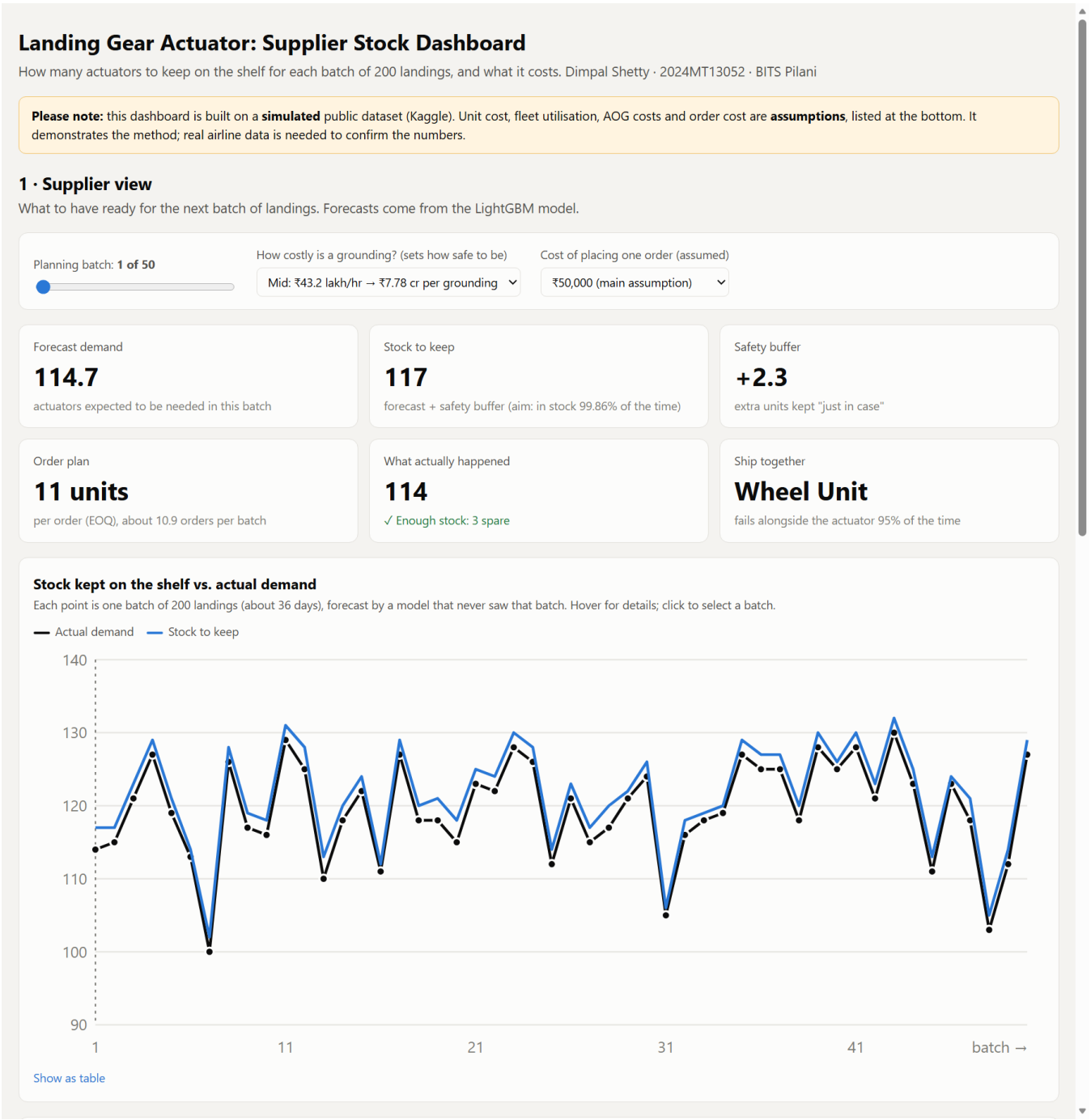


Figure 8.1: Supplier view of the dashboard (Mid scenario, batch 1).

9. Limitations

1. **No real timeline.** The data has no dates or aircraft IDs. Batches of 200 landings stand in for time periods; they are not real months.
2. **Flags are not confirmed replacements.** A flag probably means "a problem was detected", not "the part was replaced". Demand figures are therefore an upper estimate.
3. **Unrealistic demand level.** The simulated actuator flag rate of 60% gives about 119 actuators per 200 landings, far more than real aircraft would need.
4. **Assumed costs and usage.** Unit costs, the 25% holding rate, the AOG cost per hour, the 18-hour event length, the Rs. 96 per US dollar exchange rate, the Rs. 50,000 order cost and the 2,000 landings per year are all assumptions or public benchmarks, not Airbus figures.
5. **Too few periods for rare events.** Fifty batches cannot properly test a stock rule designed to run short less than once in several hundred periods.
6. **Simplified lead time.** The reorder engine assumes orders arrive before each batch starts. Real lead times for actuators can be long and uncertain.
7. **Only four parts.** Seal kits and structural bolts, named in the original outline, are not in the data.
8. **Inputs known in advance.** The forecast uses each landing's conditions. In practice, load and cycles must be estimated ahead of time from flight plans and records.

10. Conclusions and Recommendations

10.1 Conclusions

1. A complete pipeline was built, from raw landing data to a stock and order recommendation, and shown in an offline supplier dashboard.
2. The ABC/XYZ classification chose the Landing Gear Actuator as the only high-value, steady-demand part (Class A/X). This replaced the original assumption that the brake assembly was the key part.
3. In this data, landing load and landing cycles drive actuator failure, and runway condition does not. The data check showed this comes from the simulator's rules.
4. Tuned LightGBM and Random Forest models, tested fairly on unseen batches, cut the forecast error from 6.18 to 0.30 and 0.10 units per batch. Random Forest was slightly better on this data; the difference has no practical effect.
5. A cost-based stock rule turned the better forecast into a much smaller safety buffer (about 2 units instead of 23) with no stockouts in the test. This lowered the estimated yearly cost by about Rs. 1.8 to 2.5 crore, depending on the AOG scenario.
6. The most useful companion-part rule is that the wheel unit is flagged with the actuator 95% of the time, which supports bundled orders.
7. The data was found to be simulated, and this was reported openly. The main contribution is therefore the method, which can be re-run unchanged on real data.

10.2 Recommendations

1. **Test on real data.** Run the same pipeline on real airline maintenance records with dates, aircraft IDs and confirmed part replacements. This is the most important next step.
2. **Use real costs and fleet usage.** Replace the assumed landings per year, unit costs, AOG costs and order cost with real figures.
3. **Model lead time.** Add real supplier lead times, and their variation, to the reorder engine.
4. **Test over more periods.** Use a longer history to check how often the stock rule really runs short.
5. **Re-check the model choice.** Compare LightGBM and Random Forest again on real data, where the sharp simulated cut-offs will not exist.
6. **Extend to more parts.** Apply the same steps to the other parts and to other aircraft systems.

A. Settings Chosen in Each Fold

Table A.1: Settings chosen by tuning in each outer fold

Fold	Test batches (0-based)	LightGBM	Random Forest
1	13 17 19 25 26 30 32 39 45 48	15 leaves, lr 0.03, 300 trees	leaf 1, 60% features
2	3 4 6 8 12 15 37 41 46 47	7 leaves, lr 0.03, 300 trees	leaf 1, 60% features
3	0 5 9 16 24 27 31 33 34 44	7 leaves, lr 0.03, 300 trees	leaf 1, 60% features
4	1 2 11 21 23 29 35 36 40 43	7 leaves, lr 0.03, 300 trees	leaf 1, 60% features
5	7 10 14 18 20 22 28 38 42 49	7 leaves, lr 0.03, 300 trees	leaf 1, 60% features

B. Forecasts for Every Batch

Table B.1 lists, for each batch, the actual actuator flags, the forecasts of both models, the fair baseline, and the stock that LightGBM would keep in the Mid scenario (“Stock kept”). Stock kept was never below actual demand.

Table B.1: Actual and forecast actuator demand for every batch of 200 landings

Batch	Actual	LightGBM	Random Forest	Fair baseline	Stock kept
1	114	114.7	114.2	120.0	117
2	115	115.0	115.0	118.5	117
3	121	121.0	121.0	118.5	123
4	127	127.0	126.9	120.7	129
5	119	118.9	118.5	120.7	121
6	113	112.1	113.0	120.0	114
7	100	100.4	100.4	120.7	102
8	126	126.0	125.9	118.0	128
9	117	117.0	117.0	120.7	119
10	116	115.6	116.1	120.0	118
11	129	129.0	128.8	118.0	131
12	125	125.8	125.0	118.5	128
13	110	110.9	110.3	120.7	113
14	118	118.0	118.0	120.0	120
15	122	122.0	122.1	118.0	124
16	111	110.4	110.9	120.7	112
17	127	126.8	127.0	120.0	129
18	118	118.0	118.1	120.0	120
19	118	118.9	118.0	118.0	121
20	115	115.9	114.4	120.0	118
21	123	123.0	123.0	118.0	125
22	122	121.9	122.0	118.5	124
23	128	128.0	128.0	118.0	130
24	126	126.0	126.0	118.5	128
25	112	111.7	112.0	120.0	114
26	121	121.0	121.0	120.0	123
27	115	115.0	115.0	120.0	117
28	117	117.8	117.1	120.0	120
29	121	120.0	120.7	118.0	122

Batch	Actual	LightGBM	Random Forest	Fair baseline	Stock kept
30	124	124.0	124.1	118.5	126
31	105	104.0	105.0	120.0	106
32	116	116.0	116.0	120.0	118
33	118	117.1	117.6	120.0	119
34	119	118.0	119.0	120.0	120
35	127	127.0	127.4	120.0	129
36	125	124.8	124.9	118.5	127
37	125	125.0	125.0	118.5	127
38	118	118.0	118.0	120.7	120
39	128	128.1	128.2	118.0	130
40	125	124.0	125.0	120.0	126
41	128	128.0	128.0	118.5	130
42	121	121.0	121.0	120.7	123
43	130	130.0	130.0	118.0	132
44	123	123.0	123.0	118.5	125
45	111	111.0	111.0	120.0	113
46	123	122.0	122.9	120.0	124
47	118	118.7	118.0	120.7	121
48	103	103.0	103.0	120.7	105
49	112	111.9	111.9	120.0	114
50	127	127.0	126.9	118.0	129

References

[1] Bacchetti, A. and Saccani, N., "Spare parts classification and demand forecasting for stock control: Investigating the gap between research and practice", *Omega*, Vol. 40, No. 6, 2012, pp. 722-737.

[2] Lagoda, L. and Klumpp, M., "Efficient warehouse and inventory management: The modified ABC-XYZ analysis as a framework to integrate demand forecasting and inventory control", in *Dynamics in Logistics (LDIC 2024)*, Lecture Notes in Logistics, Springer, 2024.

[3] Nafiurridha and Choiruddin, A., "Classifying MRP strategy of aircraft spare parts using supervised machine learning", *Procedia Computer Science*, Vol. 234, 2024, pp. 470-477.

[4] Olmo, M. D. and Domingo, R., "Intermittent demand forecasting of aerospace rotable parts: A framework for unpredictable flight patterns", *Computers & Industrial Engineering*, Vol. 214, 2026.

[5] Stanton, I., Munir, K., Ikram, A. and El-Bakry, M., "Automation of predictive maintenance: An experimental framework for aircraft landing gear", *Engineering Reports*, 2025.

[6] Agrawal, R. and Srikant, R., "Fast algorithms for mining association rules", *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 1994, pp. 487-499.

[7] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y., "LightGBM: A highly efficient gradient boosting decision tree", *Advances in Neural Information Processing Systems*, Vol. 30, 2017, pp. 3149-3157.

[8] Grinsztajn, L., Oyallon, E. and Varoquaux, G., "Why do tree-based models still outperform deep learning on tabular data?", *Advances in Neural Information Processing Systems*, Vol. 35, 2022, pp. 507-520.

[9] Breiman, L., "Random Forests", *Machine Learning*, Vol. 45, No. 1, 2001, pp. 5-32.

[10] Silver, E. A., Pyke, D. F. and Thomas, D. J., *Inventory and Production Management in Supply Chains*, 4th edition, Boca Raton: CRC Press, 2017.

[11] Harris, F. W., "How many parts to make at once", *Factory, The Magazine of Management*, Vol. 10, No. 2, 1913, pp. 135-136, 152.

[12] Eskind, J., "Landing Gear Predictive Maintenance", Kaggle dataset (version 1, licence CC0: Public Domain), 2024, `https://www.kaggle.com/datasets/justineskind/landing-gear-predictive-maintenance`, accessed 10 September 2026.

[13] Pedregosa, F. et al., "Scikit-learn: Machine learning in Python", *Journal of Machine Learning Research*, Vol. 12, 2011, pp. 2825-2830.

[14] Raschka, S., "MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack", *Journal of Open Source Software*, Vol. 3, No. 24, 2018, p. 638.

Glossary

Term	Meaning	Page
ABC/XYZ	Ranking of parts by yearly value (A/B/C) and by how steady demand is (X/Y/Z).	15
AOG	Aircraft On Ground: the aircraft cannot fly because a part is missing.	1
AUC	A score of how well a model separates two outcomes; 0.5 is guessing, 1.0 is perfect.	22
Baseline	A simple forecast (here, a constant average) that a model must beat.	15
Batch	A group of 200 landings used as one planning period, since the data has no dates.	8
Coefficient of variation (CV)	Spread of demand divided by its average; low means steady demand.	15
Confidence / lift / support	Measures of how strongly two parts are flagged together (Apriori).	18
Cross-validation	Testing a model several times, each time on a part of the data it did not train on.	21
EOQ	Economic Order Quantity: order size that balances ordering cost and holding cost.	28
Feature importance	How much each input helped the model make its predictions.	24
Holding cost	Cost of keeping a part on the shelf (money tied up, storage, insurance).	19
Hyperparameter tuning	Trying different model settings and keeping the best one.	21
Lead time	Time between placing an order and receiving the part.	29
Log loss	A score that punishes confident wrong predictions; lower is better.	21
MAE	Mean Absolute Error: the average size of a forecast's miss.	15
MRO	Maintenance, Repair and Overhaul.	1
Newsvendor rule	A rule that sets how much stock to keep by comparing the cost of running short with the cost of extra stock.	26
Safety buffer (safety stock)	Extra units kept on top of the forecast in case demand is higher.	26
Service level	The target chance of not running out of stock in a period.	26

Term	Meaning	Page
Simulated data	Data produced by a computer program following set rules, not recorded from real events.	9